

Da imagem à classe no circuito

Inferência sequencial em SystemVerilog | sem placa e sem síntese no escopo

Este guia explica como a CNN treinada em Python foi representada com memórias, multiplicação-acumulação, comparadores e uma máquina de estados. Você pode começar com os pesos fornecidos e substituir a exportação após seu próprio treinamento.

Entrada	Operação	Saída
784 bytes 28 × 28 pixels	2 convoluções, ReLU, pooling e Dense	5 logits int32 classe 0 a 4

O que mudou em relação ao primeiro pacote

O primeiro RTL apenas contava ciclos e fixava classe zero. Esta versão contém o caminho de dados que lê pixels e pesos, calcula as camadas e escolhe o maior logit. MAC, ReLU/requantização e pooling estão efetivamente conectados ao controlador.

rtl/mac_u8s8.sv	multiplicacao + soma
rtl/relu_requant.sv	ReLU, rounding, shift, saturacao
rtl/max4_u8.sv	maximo de quatro ativacoes
rtl/tiny_cnn.sv	memorias, indices e FSM
tb/tb_arithmetic.sv	testes numericos basicos
tb/tb_cnn.sv	comparacao integral da CNN

Estado da validação. O código HDL foi implementado, mas não foi compilado ou simulado nesta sessão: o ambiente não disponibilizou um simulador. Os testes Python e o modelo escalar passaram. Eles não substituem a execução descrita nas próximas páginas.

A forma mais rápida de iniciar é usar eda_playground/trained/design.sv e testbench.sv. O primeiro possui os quatro módulos reunidos; o segundo já contém pesos, cinco imagens e todos os valores esperados.

O projeto é didático e simulável. Não há afirmação de frequência máxima, consumo, utilização de recursos ou funcionamento em uma DE10-Lite.

Primeiro teste no navegador

Dois arquivos autocontidos; sem upload dos 2.000 PNGs

1. Entre no EDA Playground e faça login. A documentação exige login para executar ou salvar um playground [1]. Selecione Verilog/SystemVerilog como linguagem e um simulador compatível; Icarus pode ser usado quando estiver disponível.

2. Abra a pasta eda_playground/trained. Copie todo o conteúdo de design.sv para o painel Design e todo o conteúdo de testbench.sv para o painel Testbench, substituindo o exemplo do site.

3. Use os dois arquivos da mesma exportação. O testbench instancia o núcleo com LOAD_FILES=0 e inicializa as memórias a partir de constantes embutidas. Não acrescente manualmente os .mem nesta variante.

4. Execute Run. Caso o simulador solicite top-level, use tb_cnn. A primeira tentativa pode revelar mensagens de compilação que ainda precisam ser resolvidas, pois não houve execução HDL aqui.

```
// Trechos de mensagens ESPERADAS se os testes passarem:  
PASS vector=0 class=... cycles=206520 logits=[...]  
...  
PASS_ALL vectors=5 exact_values_per_vector=9102 ...
```

As mensagens acima são **exemplos do formato esperado**, não um log obtido nesta sessão. Uma simulação bem-sucedida precisa terminar com PASS_ALL e não apresentar divergências ou timeout.

5. Para visualizar ondas, adicione +WAVES nas opções de execução e marque Open EPWave after run. O testbench então gera cnn.vcd; permita a janela de pop-up do visualizador [2].

6. Salve a evidência. Guarde o código e o console. O pacote preparado tem cerca de 134 mil caracteres, abaixo do limite de um milhão de caracteres documentado para o playground [1].

[1] EDA Playground — Settings & Buttons. [2] Quick Start / EPWave. Links na última página. A interface e a disponibilidade de simuladores podem variar.

Entenda módulos e interface

O controlador escolhe os operandos; o MAC é reutilizado ao longo das camadas

Módulo	Responsabilidade
mac_u8s8	Recebe ativação unsigned, peso signed e acumulador; devolve a soma do produto.
relu_requant	Zera negativos, aplica arredondamento/shift e limita a saída a 255.
max4_u8	Compara quatro bytes sem sinal e devolve o maior.
tiny_cnn	Armazena imagens, pesos e feature maps; controla índices, camadas, estado e argmax.

Como entregar uma imagem ao núcleo

Enquanto busy=0, o testbench escreve cada pixel usando input_we=1, input_addr e input_data. O endereço percorre 0 a 783 em ordem de linha. Depois dos 784 bytes, input_we volta a zero e start recebe um pulso de um ciclo.

```
endereco = linha * 28 + coluna

// Protocolo de alto nível:
escrever_todos_os_784_pixels();
dar_start_por_um_ciclo();
aguardar_done();
ler_class_id_e_logits();
```

Durante busy=1, novas escritas de pixels e pedidos de start são ignorados. done é um pulso de um ciclo ao finalizar. Nesse instante, class_id e os cinco logits representam a imagem concluída.

O núcleo não verifica sozinho que uma imagem completa foi carregada. O contrato exige os 784 pixels antes de start. O testbench implementa esse protocolo. Não alimente uma segunda imagem enquanto busy estiver ativo.

As memórias de trabalho não são zeradas a cada classificação, porque todas as posições usadas são sobrescritas na nova inferência. Não interprete os logits como válidos antes de done.

Aritmética sem surpresa de sinal

uint8 não é a mesma coisa que int8

Um pixel de valor 255 deve continuar positivo. Por isso, antes da multiplicação, o byte de entrada recebe um bit zero à esquerda e vira um valor signed de 9 bits. O peso tem 8 bits com sinal; o produto ocupa 17 bits.

```
wire signed [8:0] a_signed = $signed({1'b0,a});
wire signed [16:0] product = a_signed * b;
assign sum = acc + {{15{product[16]}},product};
```

A extensão do bit de sinal conserva produtos negativos. Por exemplo, $255 \times (-128)$ é -32.640 , não uma multiplicação de dois bytes sem sinal. O exportador verifica os limites dos acumuladores antes de gerar os arquivos.

ReLU e requantização

```
se acc <= 0: resultado = 0
senao:
  wide = acc // copia assinada de 64 bits
  se shift > 0:
    wide = (wide + 2**(shift-1)) >>> shift
  resultado = min(255, wide)
```

Caso	Resultado esperado
shift=3; acc=3	0
shift=3; acc=4	1, empate arredondado para cima
acc negativo	0 pela ReLU
resultado arredondado acima de 255	255 por saturação
shift=0	Sem deslocamento, apenas ReLU e saturação

Nesta exportação treinada, os shifts de Conv1/Conv2 são 10 e 8. Eles não estão codificados como números fixos dentro da FSM: são lidos de shifts.mem ou embutidos pelo gerador online. A Dense mantém os logits em int32 e não aplica ReLU na saída.

Memórias e ordem dos dados

Um valor por linha nos .mem; duas cifras hex para 8 bits, oito para 32 bits

Memória / arquivo	Quantidade	Largura
image_mem / images por amostra	784	8 bits unsigned
conv1_w / conv1_b	72 / 8	8 / 32 bits signed
conv2_w / conv2_b	1.152 / 16	8 / 32 bits signed
fc_w / fc_b	2.000 / 5	8 / 32 bits signed
conv1_mem / pool1_mem	5.408 / 1.352	8 bits unsigned
conv2_mem / pool2_mem	1.936 / 400	8 bits unsigned
logits	5	32 bits signed
shifts	2	8 bits unsigned

A ordem precisa ser a mesma nos dois lados

```

peso_conv[0, I, Ky, Kx]
indice_peso = ((0 * canais_entrada + I) * 3 + Ky) * 3 + Kx

feature_map[C, Y, X]
indice_ativacao = C * altura * largura + Y * largura + X

```

O primeiro peso percorre coluna do kernel, depois linha, canal de entrada e canal de saída. O flatten final percorre C, H e W. A implementação usa correlação cruzada, sem espelhar o kernel, como Conv2d [3].

No segundo pooling, o mapa tem 11 × 11. São usadas janelas com origens 0, 2, 4, 6 e 8. A linha e a coluna de índice 10 não participam. Isso reproduz o arredondamento para baixo adotado no Python.

A variante local inicializa pesos via \$readmemh. A variante online usa os mesmos valores em constantes no testbench. A diferença é a forma de carregar as memórias, não a matemática da rede.

Máquina de estados, passo a passo

Uma única rotina de produto escalar atende às duas convoluções e à Dense

Estado	O que ocorre no clock
IDLE	Aceita start quando o núcleo está livre e inicia a Conv1.
DOT_INIT	Carrega o bias no acumulador e zera o índice do produto.
DOT_MAC	Soma um produto por ciclo: 9 na Conv1, 72 na Conv2 ou 400 na Dense.
DOT_STORE	Grava a ativação requantizada ou o logit; avança a saída.
POOL	Calcula uma janela 2 × 2 por ciclo e grava seu máximo. Ao terminar, avança a camada.
ARG_INIT	Inicializa o melhor logit com a classe zero.
ARG_SCAN	Compara as classes de 1 a 4 usando >, preservando o menor índice em empate.
FINISH	Emite done, baixa busy e retorna a IDLE.

Por que separar DOT_MAC e DOT_STORE?

No último ciclo de DOT_MAC, acc recebe a soma com o último produto por atribuição não bloqueante. O valor atualizado fica disponível depois da borda de clock. Gravar o resultado em DOT_STORE, na borda seguinte, evita descartar esse último produto.

```
DOT_INIT:  acc <= bias;
DOT_MAC:   acc <= acc + produto;
DOT_STORE: saida <= requantizar(acc);
```

Contagem derivada da estrutura

```
Conv1: 5408 * (1 + 9 + 1) = 59488 ciclos
Pool1:                               1352 ciclos
Conv2: 1936 * (1 + 72 + 1) = 143264 ciclos
Pool2:                               400 ciclos
Dense:   5 * (1 + 400 + 1) = 2010 ciclos
Argmax e finalizacao:                6 ciclos
Total previsto:                      206520 ciclos
```

A carga dos 784 pixels fica fora de cycle_count. Esta é uma derivação da FSM, não uma medição HDL feita nesta sessão e não um resultado de timing de FPGA.

O que o testbench verifica

Acertar o rótulo não basta para demonstrar equivalência numérica

tb_cnn.sv carrega imagens e respostas esperadas, escreve uma imagem no núcleo, pulsa start e aguarda done com limite de ciclos. Em seguida, compara todos os elementos das quatro memórias de ativação, os cinco logits e o argmax.

Comparação por imagem	Número de valores
Saídas da Conv1 após ReLU/requantização	5.408
Pool1	1.352
Saídas da Conv2 após ReLU/requantização	1.936
Pool2	400
Logits finais	5
Classe	1
Total	9.102

A comparação usa `!=` para que X e Z não passem silenciosamente por igualdade numérica. Há conferência de arquivos, pesos não inicializados, shifts, timeout, cycle_count e pulso de done. Um erro informa a amostra e, quando aplicável, o índice da divergência.

Conjuntos de verificação prontos

Pacote	Figuras rotuladas	Estresses
exports/trained	15, três por classe	5
exports/demo	5, uma por classe	5
EDA online de cada modo	5, uma por classe	0

Os estresses são imagem zero, imagem 255, xadrez binário, impulso e ruído uniforme. Eles testam a aritmética; não são amostras válidas de uma sexta classe e não entram na acurácia.

O alvo é sempre a referência **inteira**. Mesmo que o classificador float escolha outra classe, o hardware está numericamente consistente quando reproduz os logits e os mapas inteiros esperados. Isso ainda precisa ser confirmado no simulador.

Execução local e diagnóstico

Alternativa ao navegador quando Icarus e vvp estiverem instalados

```
iverilog -V
vvp -V
python scripts/run_sim.py --export exports/trained
python scripts/run_sim.py --export exports/demo
```

O script usa iverilog para compilar com suporte SystemVerilog (-g2012) e vvp para executar. Essa separação compilador/simulador está descrita na documentação oficial [4,5]. As fontes, os parâmetros de diretório e o número de vetores são montados automaticamente.

Primeiro são executados dez testes aritméticos, depois a CNN. O script exige as mensagens de conclusão e salva logs em reports. Se as ferramentas não existirem, ele encerra com uma mensagem explícita; não realiza instalação automática.

Mensagem / sintoma	Caminho de investigação
iverilog ou vvp não encontrado	Use a variante online ou instale conforme a documentação oficial do simulador.
Arquivo ausente / memória incompleta	Confira os diretórios e a exportação inteira; não misture arquivos de treinos diferentes.
Divergência em conv1	Confira pixels /256, sinal dos pesos, bias, shift e índices do kernel.
Divergência somente em conv2	Confira canais de entrada, pool1 e os 72 produtos por saída.
Mapas iguais, logits diferentes	Confira flatten C,H,W, pesos/biases da Dense e sinal int32.
Timeout ou ciclos diferentes	Inspeccione estado, layer, out_idx, tap_idx, start, busy e done.

O clock de 10 ns do testbench é fictício. Multiplicar 206.520 ciclos por 10 ns dá 2,0652 ms de tempo simulado previsto. Isso não mede o tempo real do computador, nem garante operação física a 100 MHz.

Troque os pesos, não a lógica

Treinamento e estudo do RTL podem avançar em paralelo

O diretório trained contém parâmetros aprendidos no treinamento fornecido. O diretório demo contém uma inicialização aleatória reprodutível, sem treinamento. Não há tentativa de adivinhar pesos convergidos. Ambos percorrem a mesma arquitetura e têm referências próprias.

Após treinar na sua máquina

1. Guarde best.pt e o relatório de treinamento. **2.** Execute a exportação inteira completa. **3.** Regenere a versão online a partir dessa exportação. **4.** Rode novamente o testbench e salve o log real.

```
python -m python.export --checkpoint runs/meu_treino/best.pt \
--out exports/meu_treino
python scripts/make_eda.py --export exports/meu_treino \
--out eda_playground/meu_treino
```

No Windows, junte as linhas de cada comando. Na simulação local, altere apenas --export; no navegador, substitua os dois arquivos gerados da mesma pasta.

Pesos, biases, shifts e referências são **um conjunto inseparável**. Substituir só conv1_w.mem ou reaproveitar expected_logits.mem de outro treino invalida a comparação.

Quando é necessário editar o SystemVerilog

Somente mudar parâmetros aprendidos não exige editar a FSM. Porém, mudar tamanho de entrada, canais, kernel, padding, pooling, ordem do flatten ou classes muda a arquitetura. Nesse caso, revise dimensões das memórias, limites dos contadores, endereçamento e testbench.

O exportador deste pacote não é um conversor universal de qualquer modelo PyTorch. Ele implementa um contrato específico para esta CNN, o que torna o projeto mais simples de estudar e verificar no prazo curto.

Material para apresentar

Use o dataset e suas classes, a tabela da arquitetura, as métricas float/inteiro e o caminho dos dados pela FSM. Depois da execução HDL, acrescente o console de verificação e algumas formas de onda. Mantenha a falta de implementação física explícita.

Critérios para encerrar a etapa

Uma lista de evidências, não uma promessa de que todo o fluxo já foi validado

Evidência	Estado nesta entrega
2.000 imagens, formato e hashes	Executado e aprovado
Treinamento e avaliação float/inteira	Executados em CPU
Nove testes unitários e notebook completo	Aprovados; notebook smoke test de uma época
Modelo escalar de endereçamento	30 vetores, 273.060 valores iguais; é Python
Compilação e simulação SystemVerilog	Pendentes; simulador indisponível nesta sessão
Execução no EDA Playground	Arquivos preparados, execução pendente
Síntese e validação em FPGA	Fora do escopo

Para concluir a equivalência HDL, execute os testes aritméticos e a CNN, sem divergências ou timeout. Guarde o log com PASS_ALL, o simulador usado e os arquivos da exportação. Os resultados devem ser associados à versão exata dos pesos e do RTL.

A rede sempre escolhe uma das cinco classes. Ela não detecta posição, não rejeita automaticamente imagens vazias e não constitui um controlador de pouso. As memórias e o endereçamento priorizam clareza de simulação, não otimização para uma FPGA específica.

Referências oficiais

- [1] EDA Playground — Settings & Buttons
- [2] EDA Playground — Quick Start / EPWave
- [3] PyTorch 2.10 — Conv2d
- [4] Icarus Verilog — Getting Started
- [5] Icarus Verilog — Command Line Flags

Os estados, índices, formatos e cálculos de ciclos descritos aqui pertencem ao código entregue. A documentação externa fundamenta o uso das ferramentas e a convenção de convolução, não comprova a execução deste projeto.

Antes da submissão, confirme com a coordenação do CI Digital o formato e os documentos obrigatórios. Este pacote fornece o material técnico; não é uma verificação dos requisitos administrativos do curso.